

HelixCode session crash — headless forensic investigation

Run-id: 2026-06-13-helixcode-crash **Revision:** 1 **Last modified:** 2026-06-13T00:00:00Z **Investigator:** subagent (headless, read-only) **Binary under test:** tmux/build-darwin/bin/tmux → tmux 3.6a **Config:** scripts/tmux.conf.template **Wrapper:** scripts/tmx.template (live scripts/tmx regenerated, TMUX_BIN=build-darwin) **Platform of capture:** Darwin 24.5.0 (macOS)

Operator report (verbatim, 2026-06-13)

"Open the terminal and for terminal session choose HelixCode. It will crash the whole terminal!"

Clarified: HelixCode is a **tmx session name** (tmx new -s HelixCode or selecting it at the shell-init prompt). The crash reproduces in iTerm2, Terminal.app, a Linux terminal, AND WezTerm. HelixCode is a TUI CLI agent (Claude-Code-class) the operator runs **inside** the session.

Cross-emulator reproduction ⇒ the fault is NOT one emulator's escape handling; it is in tmx / tmux / the config, OR in operator-side runtime state that a fresh attach does not have.

Method

All captures use a Python `pty.fork()` harness that attaches a real client over a PTY and writes the **raw byte stream the outer terminal receives** to a file, then detaches with `C-b d`. A simulated HelixCode-class agent (`evidence/agent_tui.py`) requests the exact mode set Claude Code requests — `alt-screen ?1049h`, any-event mouse `?1003h`, `SGR ?1006h`, bracketed paste `?2004h` — and emits a configurable passthrough payload (`DCS ESC P tmux; ... ST + OSC-9`) to exercise allow-passthrough on.

The reattach harness performs the **exact operator path** the wrapper's attach verb performs (`scripts/tmx.template` lines 456-469): `tmux -L <sock> source-file <conf> then exec tmux -L <sock> attach`.

Evidence files live under `evidence/`. Reproduction commands are inline per hypothesis.

H1 — Pre-existing session with the agent TUI running; reattach repaints

Reproduction

```
BIN=tmux/build-darwin/bin/tmux ; CONF=scripts/
tmux.conf.template ; L=hc
"$BIN" -L "$L" -f "$CONF" new-session -d -s s -x 80 -y 24
"$BIN" -L "$L" set -g mouse on
"$BIN" -L "$L" send-keys -t s "python3 evidence/agent_tui.py
HELIXTOK 16" Enter
# fresh attach vs reattach (source-file + attach):
python3 evidence/h1_h5_attach_capture.py "$BIN" "$L" "$CONF"
fresh evidence/fresh_sink.bin
python3 evidence/h1_h5_attach_capture.py "$BIN" "$L" "$CONF"
reattach evidence/reattach_sink.bin
```

Captured evidence (mouse_any_flag=1 confirmed at attach — the Claude-Code scenario was genuinely established):

stream	bytes	ESC	DCS (ESC P)	OSC (ESC J)	ST (ESC \)
fresh_sink.bin	4200	146	0	2	2
reattach_sink.bin	4200	146	0	2	2

fresh_sink.bin and reattach_sink.bin are **byte-for-byte identical** (verified: `a == b → True`). The attach repaint is well-formed: it begins `ESC[?1049h ESC[?1h ESC= ESC[H ESC[2J ...` and ends with a normal SGR reset; no DCS passthrough is played into the outer terminal.

VERDICT: DISPROVEN as a crash cause. A reattach to a live session running the alt-screen + mouse-tracking + passthrough agent produces a bounded, well-formed repaint identical to a fresh attach. Strongest evidence: `fresh ≡ reattach` byte streams (`4200 == 4200`, identical).

H2 — allow-passthrough on replays/forwards a large passthrough payload on reattach

Reproduction (512 KB passthrough payload — 32× the H1 size):

```
"$BIN" -L "$L" send-keys -t s "python3 evidence/agent_tui.py
BIGTOK 512" Enter
python3 evidence/h1_h5_attach_capture.py "$BIN" "$L" "$CONF"
reattach evidence/big_reattach.bin
```

Captured evidence:

- `big_reattach.bin` = **3496 bytes** (SMALLER than the 16 KB-payload run, not larger).
- `DCS (ESC P) = 0` — zero passthrough DCS frames reached the outer terminal.

- The OSC-9 leak probe `b"X"*200` in `data` → **False**: the 512 KB OSC-9 passthrough payload the app emitted did NOT appear in the attach stream.

tmux paints the **current visible screen**, not the application's passthrough history. A passthrough payload of any size cannot inflate the reattach stream because it is never re-emitted.

VERDICT: DISPROVEN. Strongest evidence: a 512 KB passthrough payload yields a 3496-byte attach stream with DCS-count 0 and no payload bytes present.

H3 — extended-keys on + terminal-features 'xterm*:extkeys' emits modifyOtherKeys to the outer terminal

Reproduction (diff the attach stream with extended-keys ON vs a stripped conf with the two lines removed):

```
grep -v -e extended-keys -e extkeys "$CONF" > /tmp/conf_noext
# capture attach stream for each, scan for ESC[> ... sequences
```

Captured evidence — ESC [> ... sequences emitted to the outer terminal on attach:

conf	sequences emitted
extended-keys ON (shipped)	ESC [>c ESC [>q ESC [>4m
extended-keys OFF (stripped)	ESC [>c ESC [>q

So `extended-keys on` is solely responsible for `ESC [>4m` (CSI > 4 m = xterm **modifyOtherKeys level 4**, i.e. enable extended-key reporting). `ESC [>c` (secondary Device Attributes query) and `ESC [>q` (XTVERSION query) are unconditional and standard. The `ESC [>4m` appears in the **fresh** stream too — it is not reattach-specific. All three sequences are well-formed, balanced xterm control sequences.

VERDICT: DISPROVEN as a crash cause on the named terminals. `CSI > 4 m` is a standard, documented xterm sequence that iTerm2, Terminal.app, and WezTerm all parse without crashing. It is config- attributable but benign on the operator's emulators. Strongest evidence: the sole config-added sequence is the well-formed `ESC [>4m`, present identically in fresh and reattach streams. **UNCONFIRMED residual:** whether the operator's HelixCode TUI itself, on receiving extended-key encodings tmux forwards, emits a malformed response — this cannot be tested without the real HelixCode binary (see `diagnose.sh`).

H4 — automatic-rename-format "#{s/\.exe\$/:/pane_current_command}" mis-expands or loops

Reproduction:

```
"$BIN" -L "$L" -f "$CONF" new-session -d -s HelixCode -x 80 -y 24
"$BIN" -L "$L" display-message -p '#{s/
    \.exe$//:pane_current_command}'
# also run a process literally named claude.exe and observe
    window_name
```

Captured evidence:

- `#{s/\.exe$//:pane_current_command}` with `pane_current_command=zsh` → expands to `zsh` (rc 0). No error, no recursion.
- After the agent runs, `window_name` settles to `Python` (the strip rule correctly leaves a non-`.exe` name untouched). In the H1 capture the status bar rendered `[s] 1:Python*` cleanly.
- The format substitution is single-pass (`s/.../.../:var`); `tmux` does not re-feed the output through the format engine, so no loop is possible.

VERDICT: DISPROVEN. Strongest evidence: the format expands to a correct single value with rc 0 and the status bar renders the window name without error or runaway.

H5 — Double config application: `tmx attach` source-files the conf into a LIVE attached session

Reproduction: identical to H1 — the reattach mode of `h1_h5_attach_capture.py` runs `source-file <conf>` against the live server immediately before `attach`, reproducing `scripts/tmx.template` lines 466-468 exactly.

Captured evidence:

- `source-file` against a live server with the agent running returned exit 0, **no `stderr`** (`reattach_err.txt` empty).
- The post-`source-file` attach stream equals the no-`source-file` fresh attach stream **byte-for-byte** (`4200 == 4200`, identical — same data as H1).
- Re-sourcing the conf re-runs the `bind/set` directives; `tmux` treats redundant `bind/set` as idempotent overwrites. No directive in the conf emits bytes to the client or recurses.

VERDICT: DISPROVEN. Re-applying the config into a live session is idempotent and silent; it adds zero bytes to the attach stream. Strongest evidence: `source-file` exit 0 + empty `stderr` + byte-identical attach stream vs. the fresh path.

Determinism / re-runnability (§11.4.50 / §11.4.98)

Reattach captured N=3 consecutive times: 3496 / 3619 / 3619 bytes. Iters 2 and 3 are byte-identical. The 123-byte delta in iter 1 is a **capture-harness timing artifact**, proven by diff: the divergence at byte 1630 is the status-bar window-name field — `[s] 1:Python*` (iter 1 caught the agent as `pane_current_command`) vs `[s] 1:zsh*` (iters 2-3 caught `zsh` momentarily before the agent took the pane). The

detach trailer is clean and balanced in every iter ([detached (from session s)]). This is a property of when the harness sampled the pane, NOT a tmux non-determinism or crash.

Ranking of surviving candidates

Every headless hypothesis (H1-H5) is **DISPROVEN** as a standalone crash cause: no runaway volume, no recursive format expansion, no DCS passthrough leak, no malformed sequence forwarded to the outer terminal, fresh $\equiv$ reattach. The crash is therefore NOT in a fresh session and NOT reproducible from config + wrapper + binary state alone — it requires **operator-side runtime state** a headless attach cannot fabricate.

Surviving candidates, ranked by remaining evidentiary weight, all UNCONFIRMED — needs operator:

1. **UNCONFIRMED: the real HelixCode TUI's own output under tmux.** The simulated agent emits a *generic* alt-screen + mouse + passthrough surface. The real HelixCode binary may emit a specific escape sequence (image protocol, custom OSC, an unbalanced DCS, a huge single-line redraw) that, when forwarded via allow-passthrough on OR mis-parsed under extended-keys on, the operator's terminals choke on. This is the single highest-value unknown and is exactly what `diagnose.sh` captures from the real crash.
2. **UNCONFIRMED: a stale / wrong-arch socket or a second tmux on the operator's \$PATH.** The operator's shell-init runs `tmx attach -t NAME 2>/dev/null || exec tmx new -s NAME`. If a HelixCode socket exists from a different tmux build (e.g. system tmux vs this build) or a half-dead server, the attach behaviour is not what these captures (clean private socket) exercised. `tmux -V` of the actually-attached server + the socket inventory in `diagnose.sh` settle this.
3. **UNCONFIRMED: terminal size / \$TERM / capability mismatch at the operator's real attach.** All captures used `TERM=xterm-256color` at `80x24`. A real attach at a large size or under a different `$TERM` could trip a different repaint path. `diagnose.sh` records `$TERM`, `tput cols/lines`, and the full attach stream so the real repaint can be compared against `fresh_sink.bin`.

No likely/probably is asserted anywhere above: each VERDICT is backed by a captured byte count, exit code, or byte-identity proof; the three residuals are explicitly marked UNCONFIRMED — needs operator.

What the operator must run

`docs/qa/2026-06-13-helixcode-crash/diagnose.sh` — captures the real crashing flow read-only and prints exactly what to send back. The single load-bearing capture is the **full raw attach byte stream** written to `attach_stream.bin`; diffing it against the clean evidence/`reattach_sink.bin` shape will localise the malformed/runaway sequence the real HelixCode session emits.

Sources verified 2026-06-13

- xterm Control Sequences (CSI > Pp ; .. m modifyOtherKeys; CSI > c secondary DA; CSI > q XTVERSION): <https://invisible-island.net/xterm/ctlseqs/ctlseqs.html>
- tmux(1) allow-passthrough, extended-keys, terminal-features, automatic-rename-format: tmux 3.6a manual / OpenBSD tmux source.